

# Software Engineering Project

*Jan 2026 Term - Team 051*

*Milestone - 3*



**Submitted by:**

**Kaushik Gaur (23f1000687)**

**Shriyanshi Parashari (22f3001012)**

**Aadil Iqbal (23f1001130)**

**Nachiket Gore (23f1000542)**

# 1. Swagger-compatible YAML File

---

**Documentation Link** — [Link](#)

The API documentation for the DineFlow Restaurant Management System has been written as a fully Swagger-compatible OpenAPI 3.0 YAML file. The document covers all endpoints across the four route blueprints — Manager, Chef, Waiter, and Customer — along with complete request body schemas, response schemas, path parameters, query parameters, authentication requirements, HTTP status codes, and error responses for every endpoint. Each endpoint description includes a mapping to the relevant user story from Milestone 1 and Milestone 2 so that the connection between business requirements and technical implementation is clearly traceable. The YAML file can be imported directly into Swagger UI or Redoc to generate an interactive API reference, making it easy for any team member or evaluator to explore and test the endpoints visually without running the server.

## 2. Test Cases

---

### Introduction

This document outlines the test cases for the DineFlow Restaurant Management System. The system digitises the complete dine-in experience across four user roles — Manager, Chef, Waiter, and Customer — and exposes a REST API built on Flask with a SQLite database backend. Given the importance of ensuring reliable and seamless operation of each role-specific workflow, this document provides a comprehensive set of test cases designed to validate the functionality, correctness, and reliability of all system components, including API endpoints, order lifecycle management, inventory control, and billing operations. These test cases serve as a structured roadmap for systematically evaluating the application's capabilities and ensuring a high-quality, error-free experience for all users of the system.

### Testing Framework

The testing framework for this project is built upon Python's pytest, a robust and flexible testing tool known for its simplicity and scalability. The framework covers a wide range of tests, including unit tests to verify individual API responses, integration tests to ensure that components such as the order flow, inventory deduction, and billing pipeline work together as expected, and system-level tests to validate the complete request-to-response cycle of the application. Each test case is meticulously designed to assess specific functionalities such as HTTP status codes, JSON response structure, authentication token validation, edge cases, and error handling. The framework makes use of Flask's built-in test client, which allows all four route blueprints — Manager, Chef, Waiter, and Customer — to be tested in an isolated in-memory SQLite environment without requiring the server to be running. This structured approach ensures that the system not only meets functional requirements but also maintains correctness and stability across all API endpoints under various input conditions.

## 1 — Manager Test Cases

### 1. Authentication (/api/manager/login)

#### 1.1 Manager login with valid credentials returns HTTP 200

**Page being tested:** POST http://127.0.0.1:5000/api/manager/login

**Inputs:**

**Request Method:** POST

**JSON:** {"email": "dine@gmail.com", "password": "12345"}

**Expected Output:**

**HTTP Status Code:** 200

**JSON:** {"success": true, "token": "<jwt\_string>"}

**Actual Output:**

**HTTP Status Code:** 200

**JSON:** {"success": true, "token": "<jwt\_string>"}

**Result:** ✓ Success

```
def test_login_200(self, client):  
    res = client.post("/api/manager/login",  
                      json={"email": "dine@gmail.com", "password": "12345"})  
    assert res.status_code == 200
```

#### 1.2 No Authorization token on /inventory returns HTTP 401 [FAILED]

**Page being tested:** GET http://127.0.0.1:5000/api/manager/inventory

**Inputs:**

**Request Method:** GET

**Header:** No Authorization header

**Expected Output:**

**HTTP Status Code:** 401

**JSON:** {"error": "Unauthorized"}

**Actual Output:**

**HTTP Status Code:** 200

**JSON:** {"ingredients": [...]} // route missing auth decorator

*FIX: add @require\_manager\_token to GET /inventory route*

**Result:** ✗ Failure

```
def test_no_token_inventory_401(self, client):  
    assert client.get("/api/manager/inventory").status_code == 401  
    # FIX: add @require_manager_token to GET /inventory route
```

## 2. Tables (/api/manager/tables)

---

### 2.1 GET /tables returns HTTP 200 with tables list

**Page being tested:** GET http://127.0.0.1:5000/api/manager/tables

**Inputs:**

**Request Method:** GET

**Header:** Authorization: <valid\_token>

**Expected Output:**

**HTTP Status Code:** 200

**JSON:** {"tables": [...]}

**Actual Output:**

**HTTP Status Code:** 200

**JSON:** {"tables": [...]}

**Result:** ✓ Success

```
def test_list_tables_200(self, client, token):  
    assert client.get("/api/manager/tables",  
                      headers=auth(token)).status_code == 200
```

All Manager side test cases can be found — [Link](#)

## 2 — Customer Test Cases

### 1. GET /api/customer/tables — Customer Tables

---

#### 1.1 Customer tables — customer tables success

**Page being tested:** GET http://127.0.0.1:5000/api/customer/tables

**Inputs:**

**Request Method:** GET

**Expected Output:**

**HTTP Status Code:** 200

**Actual Output:**

**HTTP Status Code:** 200

**Result:** ✓ Success

```
def test_customer_tables_success(self, client):  
    res = client.get("/api/customer/tables")  
    assert res.status_code == 200
```

### 2. GET /api/customer/menu — Customer Menu

---

#### 2.1 Customer menu — customer menu success

**Page being tested:** GET http://127.0.0.1:5000/api/customer/menu

**Inputs:**

**Request Method:** GET

**Expected Output:**

**HTTP Status Code:** 200

**Actual Output:**

**HTTP Status Code:** 200

**Result:** ✓ Success

```
def test_customer_menu_success(self, client):  
    res = client.get("/api/customer/menu")  
    assert res.status_code == 200
```

All Customer side test cases can be found — [Link](#)

### 3 — Chef Test Cases

#### 1. GET /api/chef/queue — Chef Queue

---

##### 1.1 Get chef queue — get queue success

**Page being tested:** GET <http://127.0.0.1:5000/api/chef/queue>

**Inputs:**

**Request Method:** GET

**Expected Output:**

**HTTP Status Code:** 200

**Actual Output:**

**HTTP Status Code:** 200

**Result:** ✓ Success

```
def test_get_queue_success(self, client):  
    res = client.get("/api/chef/queue")  
    assert res.status_code == 200
```

#### 2. POST /api/chef/items/<item\_id>/done — Mark Item Done

---

##### 2.1 Mark item done — mark done success

**Page being tested:** POST http://127.0.0.1:5000/api/chef/items/{queue\_item['item\_id']}/done

**Inputs:**

**Request Method:** POST

**Setup:** queue\_item

**Expected Output:**

**HTTP Status Code:** 200

**Actual Output:**

**HTTP Status Code:** 200

**Result:** ✓ Success

```
def test_mark_done_success(self, client, queue_item):
    res = client.post(f"/api/chef/items/{queue_item['item_id']}/done")
    assert res.status_code == 200
```

All Chef side test cases can be found — [Link](#)

## 4 — Waiter Test Cases

### 1. GET /api/waiter/tables/status — Tables Status

---

#### 1.1 Tables status — tables status success

**Page being tested:** GET http://127.0.0.1:5000/api/waiter/tables/status

**Inputs:**

**Request Method:** GET

**Expected Output:**

**HTTP Status Code:** 200

**Actual Output:**

**HTTP Status Code:** 200

**Result:** ✓ Success

```
def test_tables_status_success(self, client):  
    res = client.get("/api/waiter/tables/status")  
    assert res.status_code == 200
```

## 2. GET /api/waiter/orders/ready-items — Ready Items

---

### 2.1 Ready items — ready items success

**Page being tested:** GET http://127.0.0.1:5000/api/waiter/orders/ready-items

**Inputs:**

**Request Method:** GET

**Expected Output:**

**HTTP Status Code:** 200

**Actual Output:**

**HTTP Status Code:** 200

**Result:** ✓ Success

```
def test_ready_items_success(self, client):  
    res = client.get("/api/waiter/orders/ready-items")  
    assert res.status_code == 200
```

All Waiter side test cases can be found — [Link](#)

## 3. PyTest Code

---

**Source Code Link** — [Link](#)

The pytest code for the DineFlow Restaurant Management System has been written as a single consolidated test file covering all four route blueprints — Manager, Chef, Waiter, and Customer. The file contains individual test cases organised into clearly labelled test classes, one per functional group, making it easy to run tests for a specific section in isolation. All tests use Flask's built-in test client with an in-memory SQLite database, meaning no live server or external database setup is required to run

the suite. Shared fixtures handle the complete test setup automatically — including manager login, ingredient creation, dish creation, table provisioning, order placement, batch approval, and item delivery — so each individual test remains concise and focused solely on the assertion being verified. The test file also includes edge case and negative tests for every endpoint, such as missing fields, invalid tokens, duplicate entries, non-existent IDs, and boundary conditions, ensuring that error handling in the application is validated alongside the happy-path flows.

## 4. User Feedback

---

### Feedback #1 Analytics — Add 'Items Sold' Card

Section: Manager Dashboard → Analytics    Priority: **Medium**

#### Current State

The Analytics page currently displays four summary cards at the top — Revenue, Orders Closed, Average Order Value, and a fourth card. The total number of individual dish items sold across all closed orders in the selected date range is computed by the backend analytics API and is available in the response, but it is not presented as a clearly labelled, prominent standalone card that gives the manager an instant view of kitchen throughput.

#### Requested Change

Add a dedicated "Items Sold" summary card on the Analytics page, at the same level as the existing Revenue, Orders Closed, and Average Order Value cards, so the manager can immediately see the total number of dish items sold in the selected period without having to scroll down to the detailed tables.

#### DB Change Required

**No — data already exists in the API response.**

### Feedback #2 Settings — Add Billing Control Section

Section: Manager Dashboard → Settings    Priority: **High**

#### Current State

The Settings page currently allows the manager to update basic restaurant information such as name and email. There is no dedicated billing control section that lets the manager configure what information gets printed on the customer bill, set the GST rate, add terms and conditions, or upload a payment QR code for digital payment at the table.

#### Requested Change

Add a dedicated Billing section inside Settings that gives the manager full control over everything that appears on the printed bill. And add selling price, and custom dish profit margin. The following features should be included:

#### Detailed Requirements

- Restaurant name toggle — control whether the restaurant name is printed on the bill.

- GSTIN field — allow the manager to enter and display the GSTIN number on the bill.
- Address field — allow the manager to enter and display the restaurant address on the bill.
- Phone number field — allow the manager to enter and display a contact number on the bill.
- GST rate — a configurable GST percentage field that is applied to all bill totals (e.g. 5%, 12%, 18%).
- Terms and Conditions — a text area where the manager can type custom terms that appear at the bottom of every bill.
- Payment QR code — an image upload option for a UPI or online payment QR code so customers can scan it directly on the bill for digital payment.
- Each field should have an individual toggle so the manager can choose what is shown or hidden on the bill.
- We have to add a custom dish profit margin for the customizable dish so we have to add a new column in the database in the manager table.
- We have to add selling price field in the dishes table in database.
- We have to add kitchen status and delivered at filed in the order items table in the database.

#### DB Change Required

**Yes — add gstin, address, gst\_rate, bill\_terms, custom margin column ,qr\_image\_path, selling price, kitchen status, delivered at and billing display toggle columns to the manager/settings table, order items table/dishes table .**

| # | Section   | Requested Change                                                                                                      | Priority | DB Change |
|---|-----------|-----------------------------------------------------------------------------------------------------------------------|----------|-----------|
| 1 | Analytics | Add a dedicated "Items Sold" summary card to the Analytics overview                                                   | Medium   | No        |
| 2 | Settings  | Add a Billing section — control name, GSTIN, address, phone, GST rate, terms & conditions, and payment QR on the bill | High     | Yes       |

## 5. Plan to Implement Next Sprint

Sprint 2 will build directly on the Sprint 1 deliverables. The two pieces of user feedback collected at the end of Sprint 1 — adding an Items Sold card to the Analytics section and enhancing the Billing Settings with full bill controls and waiter name on bill — will drive the code changes. Once those changes are implemented, all documentation and test cases will be updated to reflect the new API behaviour.

### Task 1a Analytics — Add Items Sold Card

- The backend analytics API (GET /api/manager/analytics) already computes and returns items\_sold in the summary object — no backend change is needed.
- Update the Analytics.js frontend component to add a clearly labelled 'Items Sold' summary card at the same level as the Revenue, Orders Closed, and Average Order Value cards.
- The card should include a descriptive sub-label and a distinct accent colour consistent with the existing card grid style.
- Verify the card value updates correctly when the manager changes the date range selector (Last 7 Days / Last 30 Days / Last 90 Days / All Time).

### **Task 1b Settings — Build Billing Control Section**

- Add a dedicated Billing tab or section inside the Manager Settings page.
- Include input fields for: Restaurant Name, GSTIN, Address, Phone Number — each with an individual toggle to control whether it appears on the bill.
- Add a GST rate input field (percentage) that is applied to all bill totals across the system.
- Add a Terms and Conditions textarea so the manager can enter custom text that prints at the bottom of every bill.
- Add a Payment QR code image upload so the manager can upload a UPI or online payment QR that is displayed on the bill for customers to scan.
- Add a live Bill Preview panel that updates in real time as each setting is changed, so the manager can see exactly how the bill will look before saving.
- Save all billing settings to the database and ensure they are applied to bills generated via GET /api/waiter/bill/<table\_number>.

### **Task 2 Update Swagger YAML Documentation**

- Update the existing dineflow\_api.yaml to reflect all API changes made for the billing settings.
- Document all new or updated fields in the GET /api/manager/settings and PUT /api/manager/settings endpoints.
- Update GET /api/manager/analytics to clearly document the items\_sold field in the summary schema with its description.
- Ensure all endpoint descriptions include error handling, user story mapping, and accurate response schemas as required by Sprint 2.

### **Task 3 Update Test Cases and Pytest Code**

- Write new test cases for the updated GET and PUT /api/manager/settings endpoints covering all new billing fields.
- Write test cases verifying that billing settings (GST rate, GSTIN, address, phone, terms, QR path) are persisted and returned correctly.
- Write test cases verifying that GET /api/waiter/bill/<n> reflects the updated billing settings in its response.
- Verify GET /api/manager/analytics returns items\_sold correctly in the summary for various date ranges.
- Update existing test cases for any endpoint whose request or response shape changed.

#### Task 4 Prepare Sprint 2 Report

- Compile all changes into a consolidated Sprint 2 report covering: changes made based on user feedback with before/after comparison, updated Swagger YAML file, backend code of implemented APIs, and updated test cases and pytest code.

### Sprint 2 Timeline (Proposed)

| Week   | Focus                                                                                             |
|--------|---------------------------------------------------------------------------------------------------|
| Week 1 | Implement all code changes (Tasks 1a, 1b, 1c), Update Swagger YAML and test cases (Tasks 2 and 3) |
| Week 2 | Testing, bug fixes, and Sprint 2 report preparation (Task 4)                                      |

Usage of AI has been done in the backend and we have uses Codex extension in VS code , screenshots can be found here - [Link](#)

Full application code can be found here - [Link](#)

---